

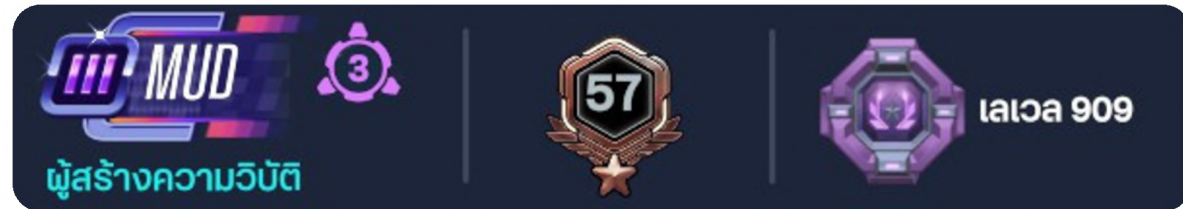
Physical Aimbot for FPS Games using Computer Vision

Pollapaat Suttimala – ID: 65340500046

Advisor: Dr. Suriya Natsupakpong

Why build an Aimbot Robot?

- เป็นคนชอบเล่นเกม FPS เป็นชีวิตประจำวัน โดยเฉพาะเกมที่ใช้ความแม่นยำสูง



- เกิดคำถามว่า “ถ้าหุ่นยนต์มาเล็งแทนเราได้จริง มันจะทำได้แม่นยำแค่ไหน?”

พัฒนาระบบจริงที่จับต้องได้ ซึ่ง ไม่ใช่แค่โค้ดยิงเป้าในเกม แต่คือระบบที่ **ตรวจจับด้วยภาพจริง → ควบคุมหุ่นจริง → เลื่อนเมาส์จริง**

และที่เลือกใช้แบบ “Physical Aimbot” แทนการใช้ซอฟต์แวร์ เพราะ:

- ✓ เพื่อหลีกเลี่ยงระบบ Anti-Cheat ในเกมที่อาจมองว่าเป็นการแฮ็ก
- ✓ และเพื่อให้ระบบสามารถนำไปใช้งาน หรือทดสอบในเกมจริงได้ในอนาคตโดยไม่มีผลกระทบเชิงความปลอดภัย

- ใช้ Kovaak เป็นสนามทดสอบ เพราะมันมี Scenario หลากหลายที่ทำทนายระดับโปร อีกทั้งยังไม่รบกวนผู้เล่นอื่นในเกมจริง

แม้จะทำลายค่าเฉลี่ยคะแนนของผมที่พยายามมาหลายปีไปเยอะมากก็ตาม

- เริ่มจากความชอบ แต่กลายเป็นโอกาสในการทดลองและเรียนรู้ระบบ Computer Vision + Robotics แบบ end-to-end

“This became the perfect opportunity to test how far CV+Robotics could actually go — in a domain I truly care about.”

Project Objective

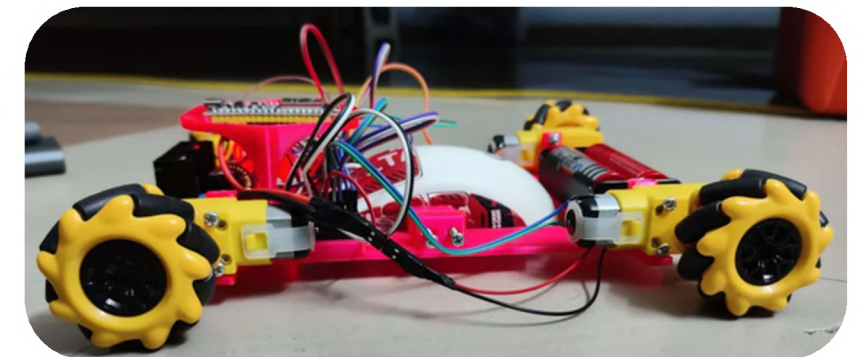
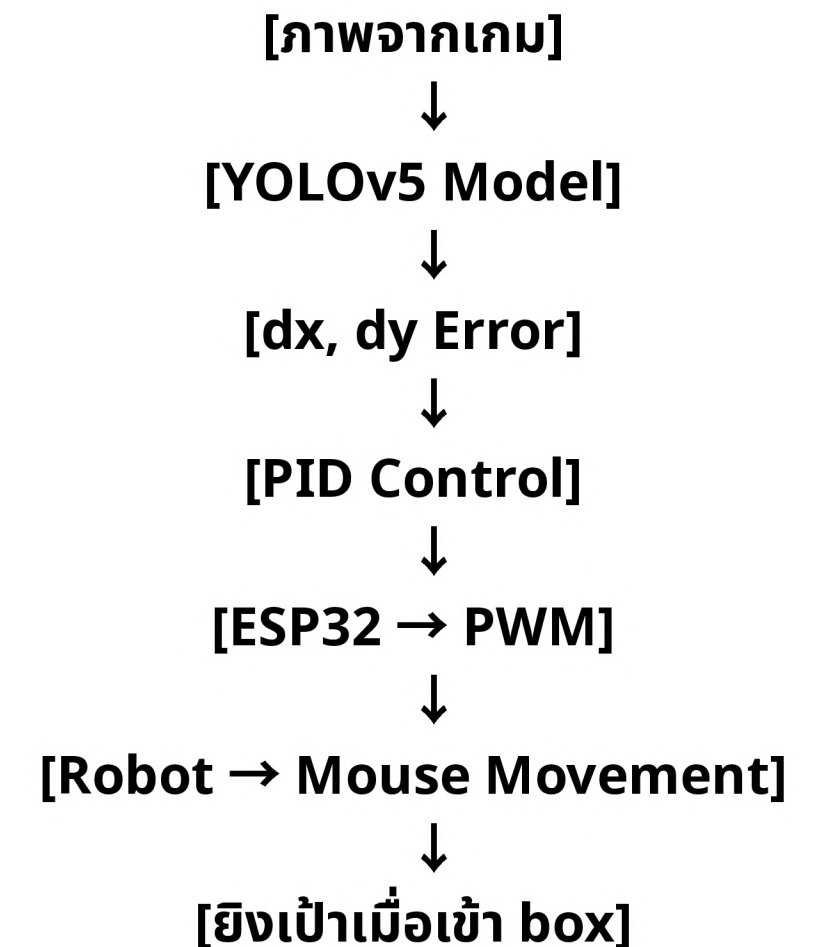
พัฒนา “Physical Aimbot” ที่สามารถตรวจจับเป้าจากจอเกม และควบคุมเมาส์จริงด้วยหุ่นยนต์ เพื่อเล็งเป้าโดยอัตโนมัติ โดยใช้การผสมผสานระหว่าง Computer Vision และ Robotic Control (PID + ESP32 + Mecanum) เป้าหมายคือ เข้าใจปัญหาการควบคุมจริง และ สร้าง baseline สำหรับระบบช่วยเล็งที่ต่อยอดได้ในอนาคต

System Outputs

- หุ่นยนต์ที่เลื่อนเมาส์จริงตามตำแหน่งเป้าหมาย (Mouse-moving Robot Prototype)
- โมเดล Object Detection (YOLOv5) ที่ตรวจจับเป้าในเกมแบบเรียลไทม์
- โครงสร้างควบคุมแบบ **Vision Based**: ภาพในเกม → ตรวจจับ → dx,dy → PID → เคลื่อนเมาส์ → ยิง

Engineering Scope & Focus

- ใช้ Kovaak Aim Trainer เป็น environment เพื่อไม่รบกวนผู้เล่นอื่น + ได้ค่าประเมินชัด
- ใช้ Vision-based control แทน Encoder → ทดสอบการควบคุมที่พึง “การมองเห็นล้วน ๆ”
- ใช้ Hardware ราคาประหยัด (TT Motor, ESP32) → รับมือกับข้อจำกัดเชิงกลไกจริง
- ทดสอบด้วยคะแนน Accuracy / Score จากเกม → เพื่อวัดผลลัพธ์ที่เชิงสถิติชัดเจน
- โครงการนี้ไม่ใช่เพื่อโกงเกม แต่เพื่อศึกษาการผสาน CV + Robotics เชิงวิศวกรรม



Scope & Timeline Overview

Work Scope (ขอบเขตของงาน)

✅ ขอบเขตที่ครอบคลุมในโครงการนี้:

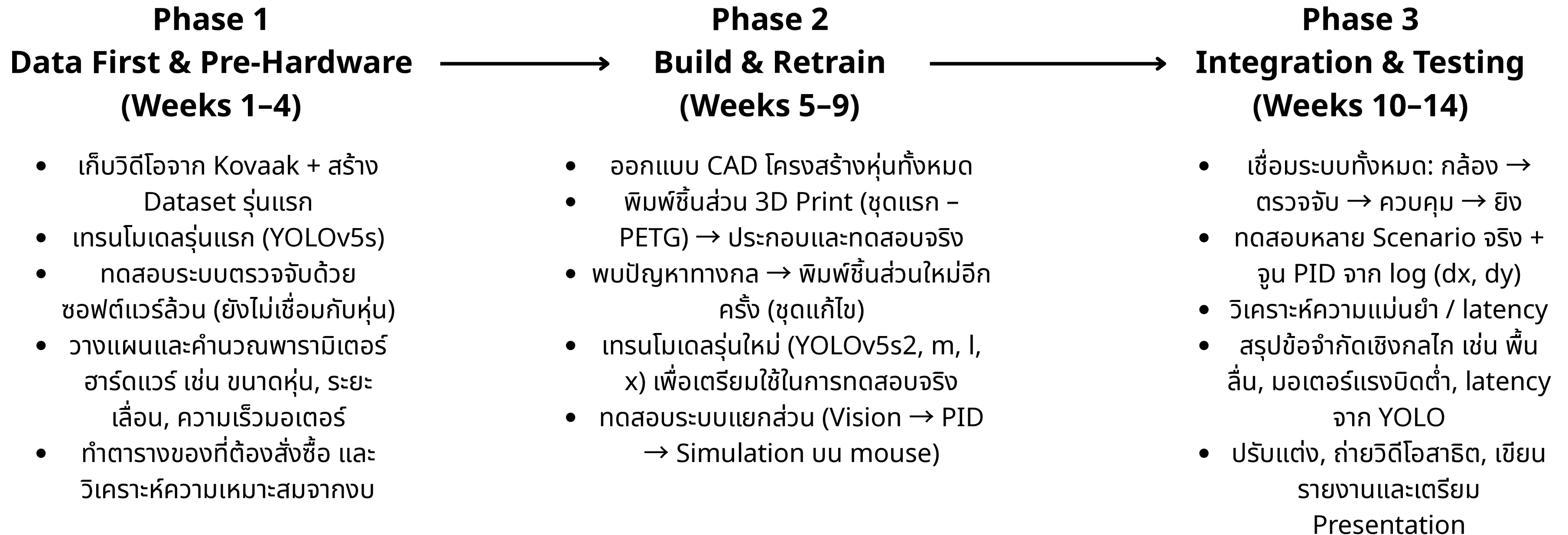
- ใช้เกมฝึกเล็ง Kovaak เป็นสนามทดสอบหลัก
- ตรวจจับเป้าหมายแบบ real-time จากหน้าจอเกม
- ควบคุมหุ่นยนต์เลื่อนเมาส์จริง ด้วยมอเตอร์ Mecanum
- เชื่อมระบบตรวจจับ + PID + ESP32 + mouse movement แบบครุ loop
- ประเมินผลด้วยคะแนนและความแม่นยำจาก Kovaak

⚠️ สิ่งที่ยังไม่รวมในการทดลอง:

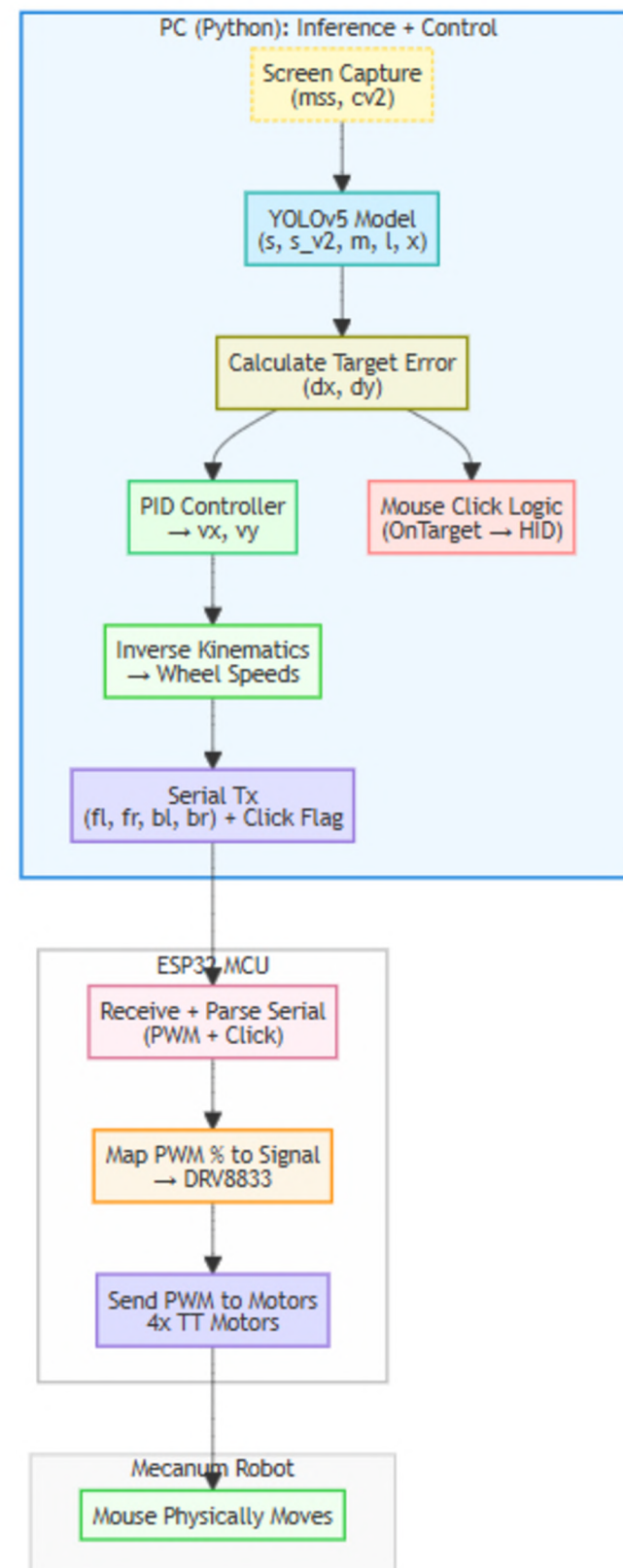
- ยังไม่เน้นเก็บ latency แบบละเอียด
- ยังไม่ใช้งานกับเกม FPS จริงแบบ competitive
- ยังไม่พัฒนา module คลิกระบบเต็มระบบ (แต่มี logic รอไว้แล้ว)

Scope & Timeline Overview

Timeline Summary



System Architecture Diagram



- ✓ ระบบควบคุมเป็น **closed-loop แบบเห็นภาพ** (vision-based feedback)
→ **ไม่มี encoder**, อิงเฉพาะ **bounding box** จาก **YOLOv5** เท่านั้น
- ✓ ตรวจจับเป้าหมายด้วย **YOLOv5** → คำนวณ ตำแหน่งเป้าหมาย **dx, dy**
→ ป้อนเข้าสู่ **PID** เพื่อคำนวณ ความเร็วในแกน **vx, vy**
- ✓ จาก **vx, vy** → ใช้ **Inverse Kinematics** สำหรับ **Mecanum** เพื่อแปลงเป็น ความเร็วล้อ 4 ตัว
→ ส่งค่า **PWM** ผ่าน **PySerial** ไปยัง ESP32 เพื่อควบคุมจริง
- ✓ การคลิกใช้ **logic** จากฝั่ง **PC**: ยิงเมื่อเป้าเข้าสู่ **target box** (Mode 1: track , Mode 2: pulse-click)
→ วางแผนพัฒนาเป็น **HID Bluetooth** ต่อในอนาคต
- ⚠ ระบบ**ไม่มี feedback** ตำแหน่งจริง
→ หาก YOLO **ไม่ตรวจเจอ**เป้า → PID ไม่ได้ input → **หุ่นนิ่ง** (ไม่แกว่ง แต่ไม่เล็งด้วย)

Design Phase

ระบบตรวจจับเป้า + การควบคุมหุ่นยนต์ + PID + Integration

ทำไมต้อง YOLO

◆ ปัญหาที่ต้องแก้

- ระบบต้อง ตรวจจับตำแหน่งเป้าหมายจากหน้าจอเกมแบบ real-time
- เป้าในเกมเคลื่อนไหวเร็ว, ซ้อนกัน, และ “เกิดใหม่” ได้ตลอด
- ไม่สามารถใช้ image classification ได้ → ต้องใช้ object detection ที่มี bounding box

◆ เหตุผลที่เลือก YOLOv5

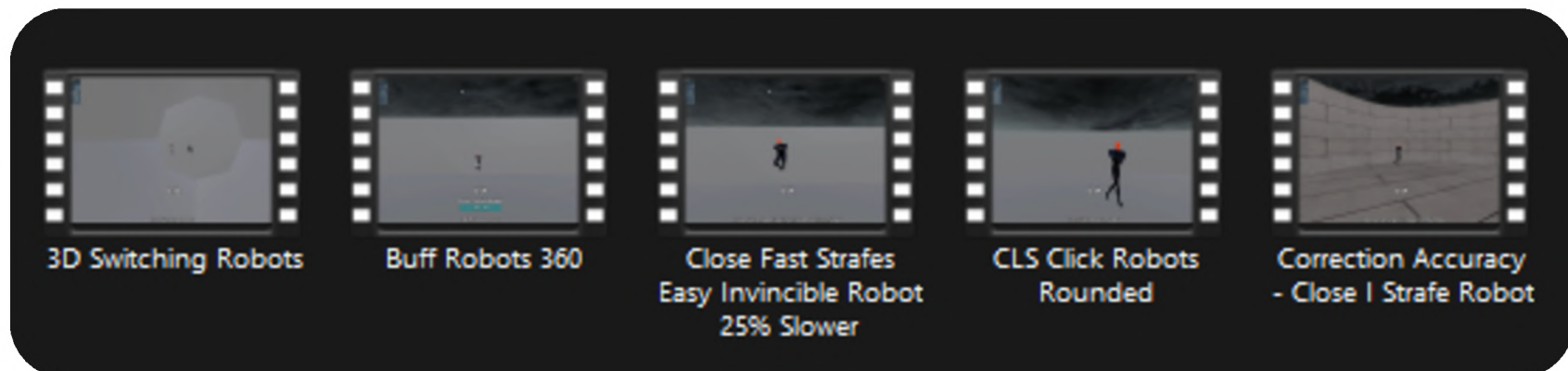
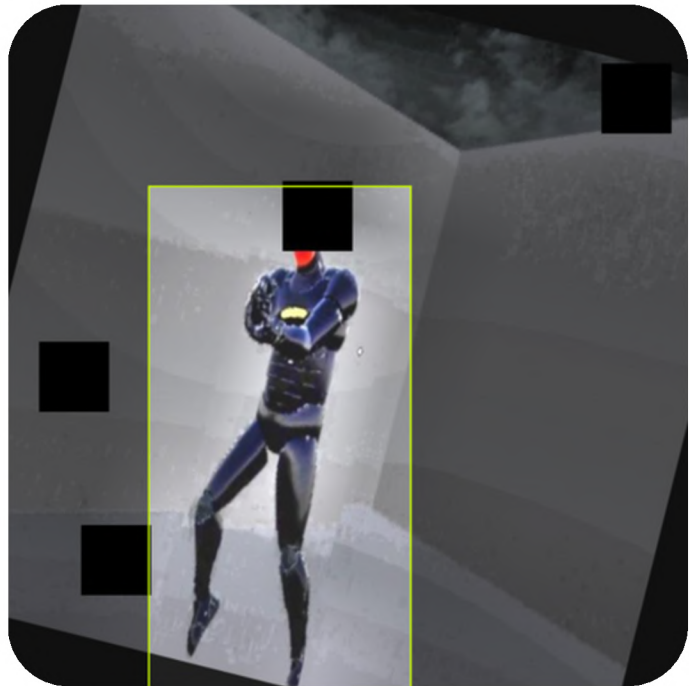
- ตรวจจับได้ frame ต่อ frame เหมาะกับเป้า dynamic ในเกม
- มีหลายขนาดให้เลือก (s, m, l, x) → ปรับตาม latency/hardware
- YOLOv5 ถูกใช้ในงาน real-time vision มากมาย
- ใช้งานบน PyTorch ได้โดยตรง
- ให้ output เป็น bounding box (x, y, w, h) → ใช้คำนวณ dx, dy ได้ทันที

◆ ทำไมไม่ใช้ Object Tracking?

- Object tracking เช่น CSRT / KCF ต้องเริ่มจาก frame แรก → ไม่รู้จักเป้าใหม่ที่โผล่มาทีหลัง
- เป้าในเกม “เกิดใหม่ / หายไป / teleport” → ต้อง detect ใหม่ตลอด
- Tracking ไม่สามารถรับมือ multi-target ได้ดี → ใช้ได้เฉพาะกรณีเป้าหนึ่ง

“YOLOv5 คือโมเดลที่ balance ได้ดีที่สุดระหว่าง ‘ความเร็ว’ และ ‘ความแม่นยำ’ เหมาะกับระบบของเราที่ต้อง detect เป้าแบบต่อเนื่อง และนำตำแหน่งเป้า นั้นมาใช้ควบคุมเม้าส์จริง”

Dataset รุ่นแรก: จุดเริ่มต้นของโมเดล



```
val: Scanning D:\UNIVERSITY\YR3\FRA361_Open_Topic\DATASET\For Train\FRA361_OpenTopic_Aimbot.v1i.yolov
Class  Images  Instances  P      R      mAP50  mAP50-95: 100%|
all    1042      2640      0.933  0.802  0.891  0.504
Speed: 0.3ms pre-process, 4.8ms inference, 1.7ms NMS per image at shape (32, 3, 640, 640)
Results saved to runs\val\exp
```

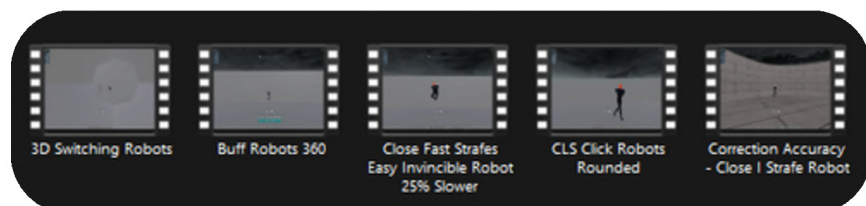
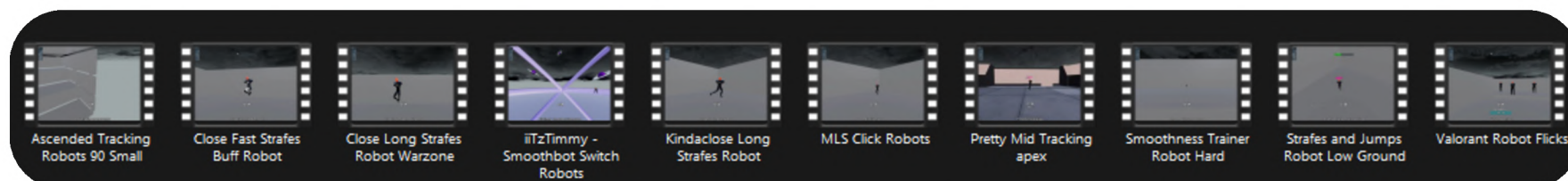
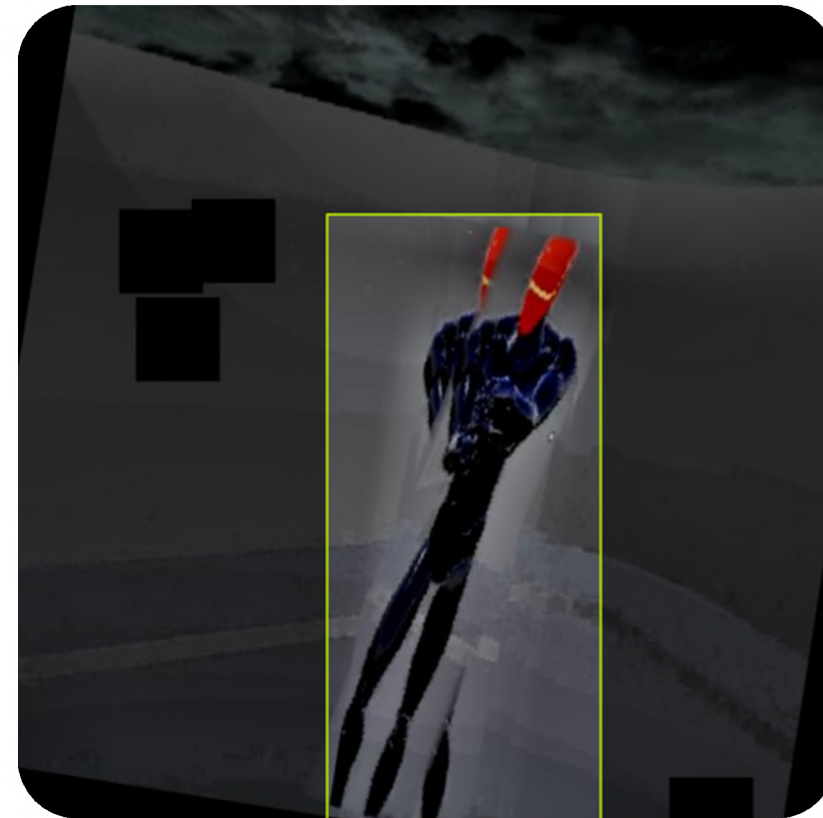
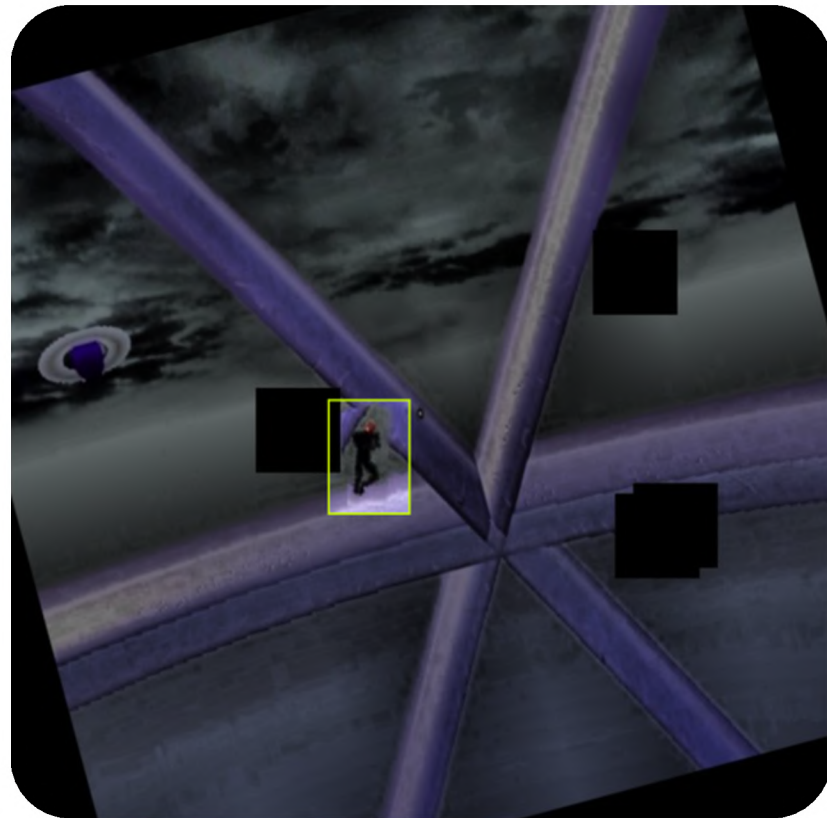
- ขั้นตอนการสร้าง Dataset V1:
บันทึกจาก Kovaak จำนวน 5 Scenarios รวม ~5 นาที
แยก frame → สร้าง bounding box ด้วย Roboflow
ใช้ Augment พื้นฐาน:
 - Horizontal/Vertical Flip (ความหลากหลายของการเคลื่อนที่หุ่น)
 - Random Crop (กรณีหุ่นอยู่หลังอุปสรรคต่างๆ)
 - Shear ($\pm 10^\circ$) (เมื่อหุ่นมีการเคลื่อนไหวในกรณีจับภาพกับกล้อง)
 ภาพทั้งหมดหลัง augment $\approx 11,342$ รูป

- จุดอ่อนที่พบ:
Augment ยังไม่พอกหลากหลาย → ไม่ generalize
Scenario เดิม ๆ → overfit ใน training set
เริ่มเห็น false negative ชัดเมื่อเอาไปใช้ในฉากใหม่

- ผลเทรน YOLOv5s รุ่นแรก:
mAP@0.5: 0.892
Precision: 0.932
Recall: 0.804

ใช้ เครื่องส่วนตัวในการ Train
CPU: Ryzen 7 5800H
GPU: NVIDIA RTX 3060 (6GB VRAM)
RAM: 16GB

Dataset รุ่นสอง: ขยายเพื่อแก้ปัญหา Overfit



V2: ~38,409 รูป

ปัญหาจาก V1 ที่เจอ

- โมเดล YOLOv5s จาก V1 แม่นเฉพาะฉากที่คล้าย training set
- ไม่สามารถ generalize ไปยังฉากจริงที่มี dynamic หรือแสงต่างได้
- Augment ยังหลากหลายไม่พอ → เกิด False Negative สูง
- GPU ที่ใช้ (RTX 3060) เริ่มฝึก YOLOv5m ไม่ไหว (CUDA crash / VRAM เต็ม)

วิธีแก้: Dataset V2 + เปลี่ยนเครื่องเทรน

- เพิ่ม Scenario → ครอบคลุม blink, click-flick, low-light, overlap
- ลด Sampling เหลือ 5 fps → ได้ภาพไม่ซ้ำ ลด redundancy
- ตรวจ label ใหม่หมด (แก้ miss-label)
- เพิ่ม Pre-processing:
 - Auto-orientation, adaptive contrast, resize
- เพิ่ม Augmentation ใหม่:
 - Horizontal flip, rotation, brightness shift, salt-pepper noise, box crop, blur

ย้ายการฝึกไปใช้ Server (Tesla V100 ×2 GPU)

- เทรนได้ batch ใหญ่ขึ้น + เร็วกว่าหลายเท่า
- รองรับเทรน model ขนาดใหญ่ขึ้น (m, l, x)
- แยก GPU สองตัวรันพร้อมกัน ลด wait time

ผลลัพธ์

- Dataset ใหม่ขยายจาก 11K → 38K ภาพ
- ความหลากหลายของภาพสูงขึ้นมาก
- เทรนได้หลายขนาดของ YOLOv5 พร้อมกัน
- เตรียมพร้อมสำหรับการเลือก model ที่เหมาะกับ hardware จริงในการใช้งาน

Training V2: YOLOv5 ทุกขนาดบน GPU V100

ตารางผลการเทรน

Model	Precision	Recall	mAP@0.5	Latency (ms) → from Tesla V100
YOLOv5s_v2	0.885	0.652	0.789	2
YOLOv5m	0.843	0.602	0.747	4.7
YOLOv5l	0.821	0.681	0.761	8.2
YOLOv5x	0.844	0.681	0.77	14.6

- 🧠 เป้าหมายการเทรนรอบใหม่:
- เทรนใหม่ทั้งหมด เพราะ:
- ⚠️ ไม่ใช้ weight จาก Windows → Linux ไม่ได้ (posix path error, model.ema keyerror)
 - ⚠️ ไม่ continue training บน GPU V100 เพราะ train.py ต้องแก้ → กระทบ model อื่นที่รันคู่กัน
 - ✅ เริ่ม train ใหม่ทุก model

- 💻 Hardware Training Spec:
- Tesla V100 × 2 (แยก GPU รับ 2 model พร้อมกัน)
 - Train แบบ full-size + full-epoch
- 🔧 รายละเอียดการฝึก:
- YOLOv5s_v2 → batch 32 → early stop @109
 - YOLOv5m → batch 24 → early stop @112
 - YOLOv5l → batch 16 → early stop @58
 - YOLOv5x → batch 12 → early stop @59
 - (เปิด option: --optimizer adam, --sync-bn, --rect, --patience 50)

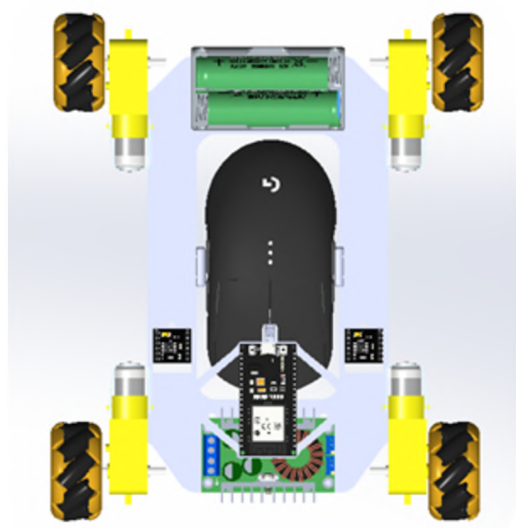
- ⚙️ การใช้ resource:
- ใช้ --device 0 และ --device 1 เพื่อแยก load ระหว่าง GPU
 - ใช้ --rect เพื่อลด VRAM load

- สรุปจากค่า Valid
- YOLOv5s_v2 = ดีสุด balance เร็ว+แม่น
 - YOLOv5x = แม่นสุดแต่ latency สูง
 - YOLOv5m = เหมาะใช้จริงกับ PID

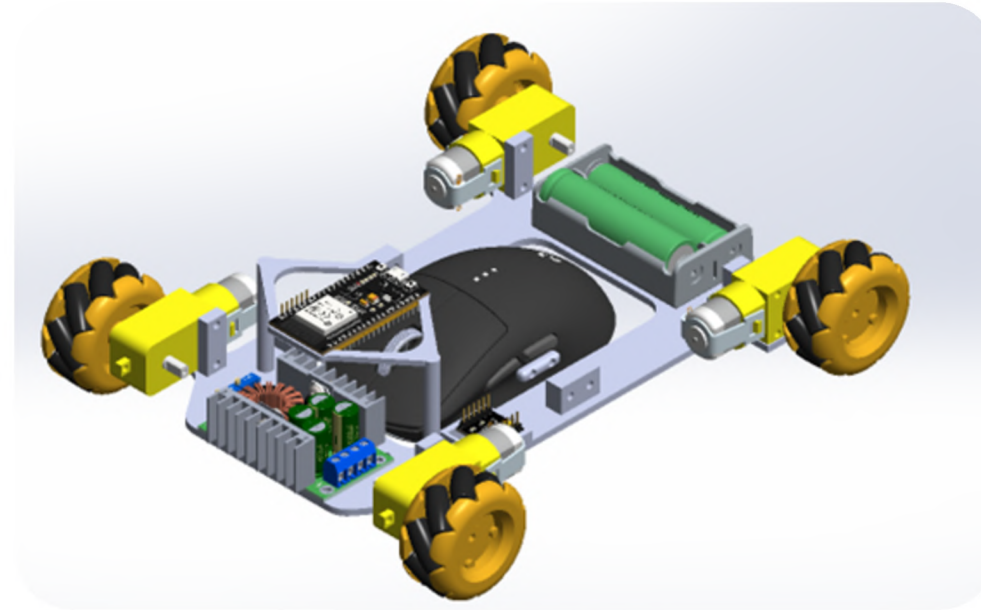
CAD Design & Layout

ออกแบบหุ่นจากศูนย์: วางแผนสองชั้น + จัดวางอุปกรณ์

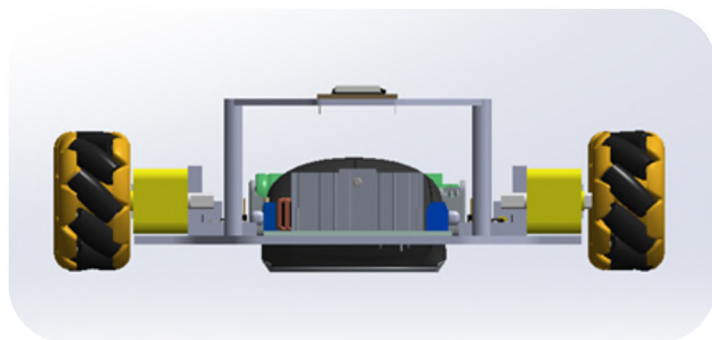
ภาพ CAD จาก SolidWorks:



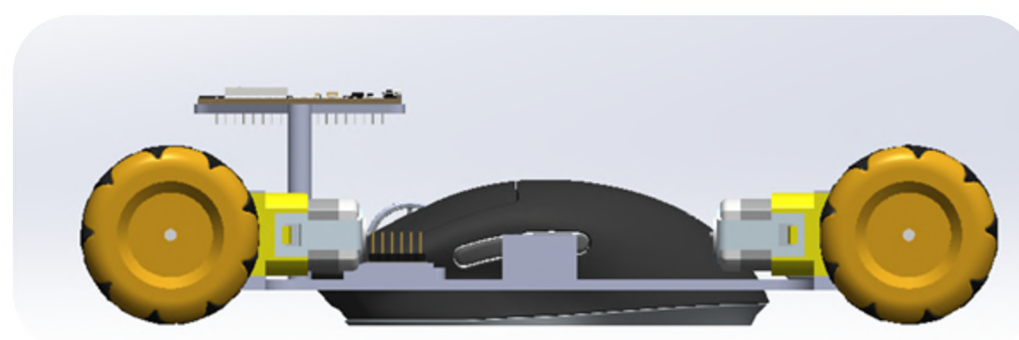
Top View



Isometric View



Front View



Side View

ออกแบบด้วยโปรแกรม SolidWorks

โครงสร้างแบบ 2 ชั้น:

- ชั้นล่าง: วางมอเตอร์, ล้อ, XL4016, Battery
- ชั้นบน: วาง ESP32, เดินสายไฟให้ไม่ชนกับส่วนเคลื่อนที่

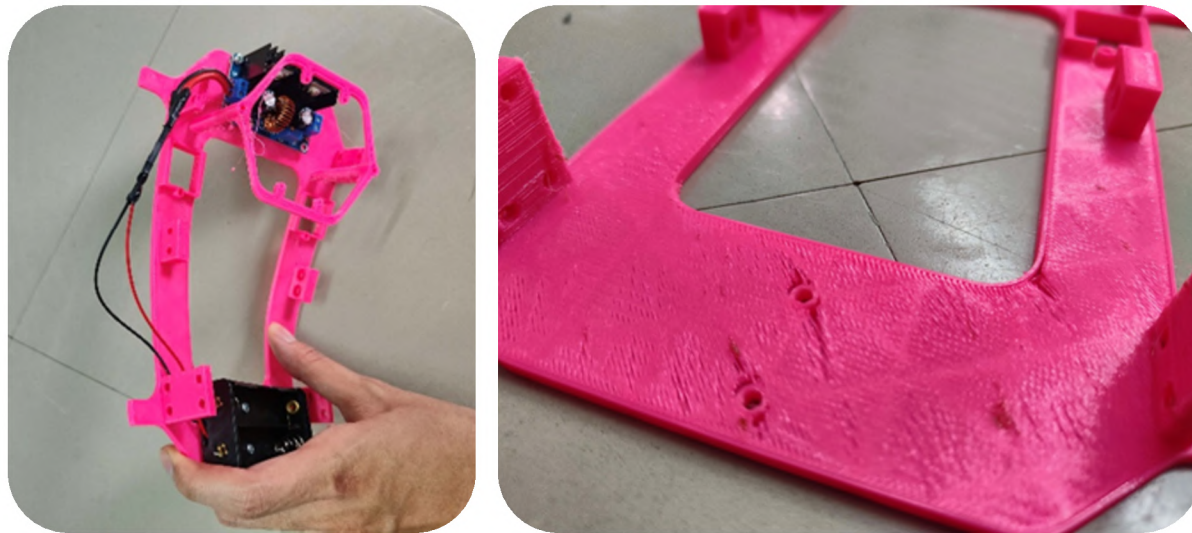
ตำแหน่งอุปกรณ์:

- มอเตอร์ตรงมุม เป็นสี่เหลี่ยมจตุรัส 20x20 cm. → คำนวณ Inverse Kinematics ง่าย
- ESP32 แยกไว้ชั้นบน → เดินสาย USB → PC ง่าย, ไม่รบกวนสาย PWM
- Mouse Lock อยู่ตรงกลาง → จับเมาส์ได้แน่น

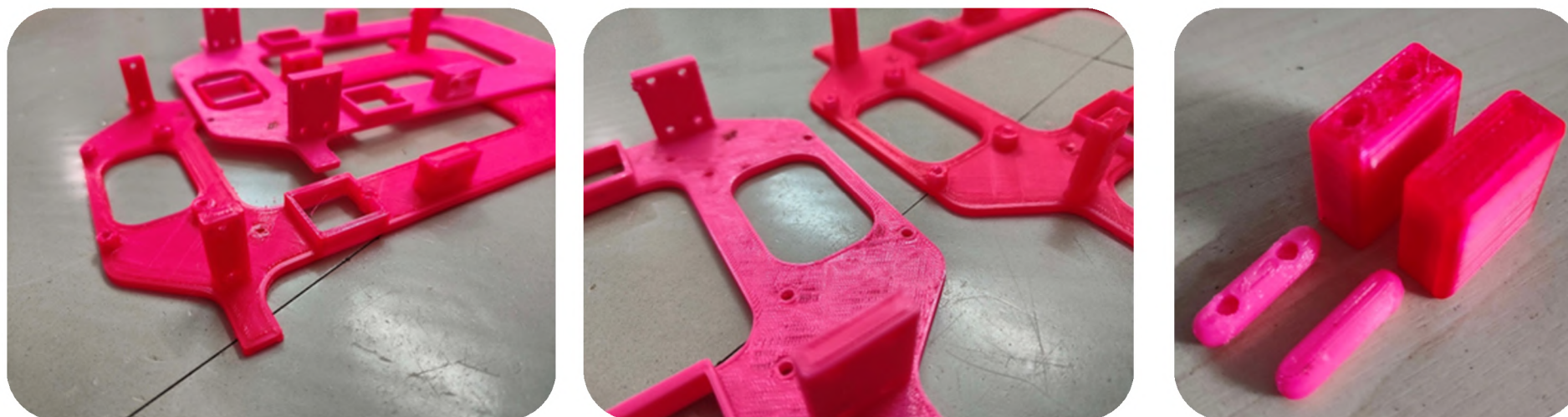
3D Print Iteration

พิมพ์เสีย → ปรับ CAD → พิมพ์ใหม่จนใช้ได้จริง

ภาพอุปกรณ์จากการพิมพ์ครั้งแรก



ภาพเปรียบเทียบระหว่างครั้งแรกและปัจจุบัน *before สีชมพู after สีแดง



⚠ ปัญหาที่เจอในการพิมพ์ครั้งแรก

- ใช้ PETG ครั้งแรก → เครื่องยังตั้งไม่พอดี + thermistor มีปัญหา
- บางชิ้น “อ่อนตัว-ย้วย” เหมือนสปริง → หุ่นโยก ไม่มั่นคง
- Mouse Lock ขนาดเล็กเกินไป ยึดเมาส์ไม่อยู่
- XL4016 ยื่นขาบัดกรีชนโครง → ยึดไม่แน่น

🔧 แนวทางแก้ไข (ปรับ CAD แล้วพิมพ์ใหม่)

- ปรับตำแหน่งรูยึดมอเตอร์ → ให้พอดีกับมอเตอร์จริง
- เพิ่ม “Spacer” รองขาบัดกรีของ XL4016
- ขยายขนาด Mouse Lock + ล็อกช่องนิ้ว + ทำให้ใส่ง่ายขึ้น

⚡ จูนแรงดัน XL4016 → 7.77V

- เดิมใช้ 7.4V → แรงบิดไม่พอ(มอเตอร์ Torque ต่ำ) โดยเฉพาะเวลาสตาร์ท
- ปรับเป็น 7.77V (6-8V ในรายละเอียดที่ร้านเขียนไว้)
- 🎩 “เลือก 7.77 เพราะรู้สึกเหมือนเลขแจ๊คพ็อต + สมดุลดีในเชิงจิตวิทยา”

✅ ผลลัพธ์หลัง Iteration

- หุ่นแข็งแรงขึ้น เคลื่อนได้จริง ใช้งานทดสอบได้
- ทุกอุปกรณ์ยึดแน่น ไม่โยก ไม่หลุด
- โครงสร้างพร้อมสำหรับ Integration กับระบบควบคุม

Theory & Hardware Justification

เหตุผลการเลือกอุปกรณ์ และแนวทางการควบคุม

🌀 ทำไมเลือก Mecanum Wheels?

- ล้อ **Omni** เคลื่อนที่ holonomic ได้ แต่ **แพงกว่า 4-5 เท่า**
- Mecanum** ใช้ลูกกลิ้งเอียง → เคลื่อนที่ทุกทิศทางได้เช่นกัน
- ราคาถูกกว่า แต่ยังให้ประสิทธิภาพ**เพียงพอ**ในการควบคุมเม้าส์
- ควบคุมซับซ้อนกว่าเล็กน้อย ต้องใช้ **Inverse Kinematics**

🔺 Inverse Kinematics สำหรับการเล็งเป้า

- จากค่าที่ต้องการเคลื่อน (V_x, V_y, ω) → คำนวณความเร็วล้อทั้ง 4
- สมการ (สำหรับ 4 ล้อ Mecanum):

$$V_{FL} = V_x - V_y - (L + W) * \omega$$

$$V_{FR} = V_x + V_y + (L + W) * \omega$$

$$V_{RL} = V_x + V_y - (L + W) * \omega$$

$$V_{RR} = V_x - V_y + (L + W) * \omega$$

FL: Front Left (หน้าซ้าย)
FR: Front Right (หน้าขวา)
RL: Rear Left (หลังซ้าย)
RR: Rear Right (หลังขวา)
 V_x (Linear Velocity - X-axis)
 V_y (Linear Velocity - Y-axis)

กรณีนี้ $\omega = 0 \rightarrow$ ไม่ต้องหมุน

- ใช้แปลงคำสั่ง “เล็ง” → การหมุนของล้อ → ขยับเม้าส์จริง

🎯 PID Controller แบบไม่มี Feedback

- ไม่มี Encoder → ใช้ **dx, dy** จาก **bounding box** YOLO เป็น error แทน
- ใช้ PID แยกแกน:
 - X-axis → ซ้ายขวา
 - Y-axis → เดินหน้า ถอย
- จูนแบบ Data-Driven จาก log จริง (aimbot_log.csv)
“เป็น closed-loop ที่ feedback มาจากภาพ ไม่ใช่จาก encoder”

⚙️ วิเคราะห์ มอเตอร์ Yellow TT (1:48)

- จาก datasheet: Torque ~ 0.147 Nm
- รัศมีล้อ = 24 mm $\rightarrow F = \tau / r \approx 6.125$ N ต่อล้อ
- ใช้ 4 ล้อ $\rightarrow$ แรงรวม ~ 24.5 N
- คำนวณแรงต้านจากพื้น:
 - น้ำหนักหุ่น: ~ 0.5 kg
 - $\mu \approx 0.4 \rightarrow F_{\text{friction}} \approx 1.96$ N
 - แรงขับเพียงพอทางทฤษฎี

⚠️ ข้อจำกัดจากการใช้งานจริง:

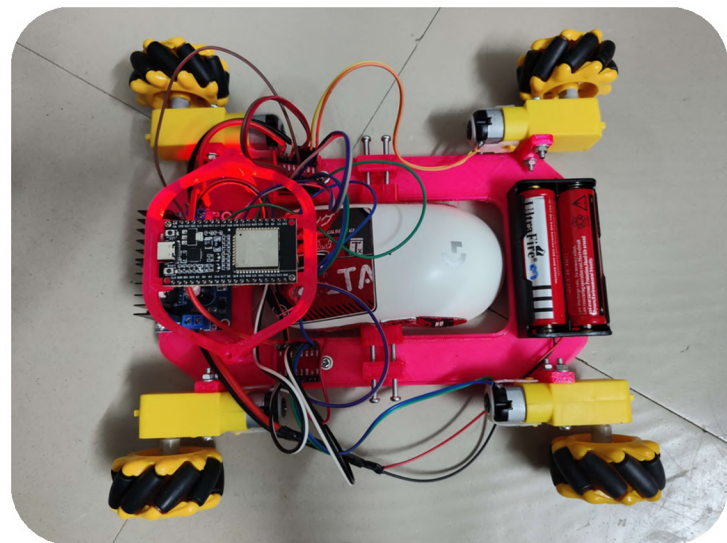
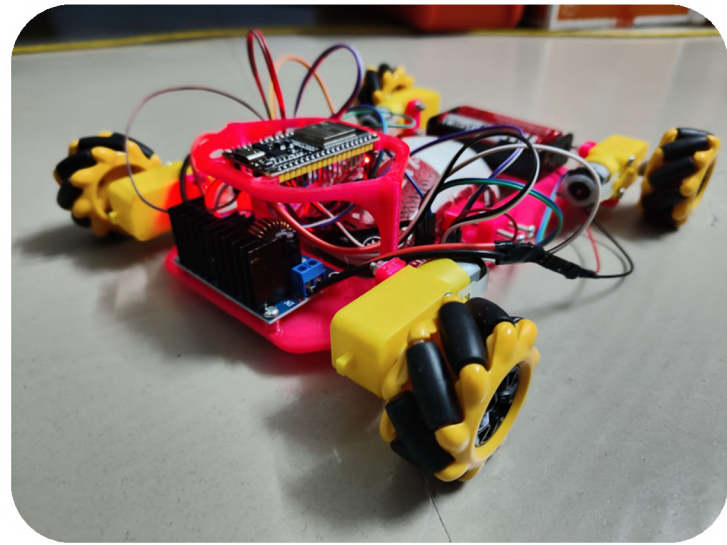
- Deadzone: PWM < 145 ล้อไม่หมุนเลย
- กระชากตอน start $\rightarrow$ ส่งผลต่อ PID tuning
- ขยับละเอียดไม่ได้ในความเร็วต่ำ $\rightarrow$ เคลื่อนที่หยاب

🧠 ข้อเสนอแนะในอนาคต

- พิจารณาใช้มอเตอร์แบบ มี encoder
- หรือเปลี่ยนไปใช้ Stepper / Servo เพื่อควบคุมแม่นยำขึ้น
- เพิ่ม feedback loop $\rightarrow$ ควบคุมได้ละเอียดขึ้น

Mobility Test & Wiring

ทดสอบการเคลื่อนที่ทุกทิศทาง + Wiring คสว



Complete Assembly



Move Test Footage

การเชื่อมต่อ ESP32 → DRV8833

ESP32 Pin	DRV8833	Motor
GPIO 25, 26	Driver 1	ล้อหน้าซ้าย
GPIO 27, 14	Driver 1	ล้อหน้าขวา
GPIO 12, 13	Driver 2	ล้อหลังซ้าย
GPIO 32, 33	Driver 2	ล้อหลังขวา

🚗 ทดสอบการเคลื่อนที่ 8 ทิศทาง

- เคลื่อนที่พื้นฐาน:
 - เดินหน้า / ถอยหลัง
 - สโตรฟซ้าย / ขวา
 - หมุนซ้าย / ขวา (rotate)
 - เคลื่อนที่เฉียง (diagonal)
- ใช้คำสั่ง `digitalWrite()` กับ `delay` ควบคุมทิศทางละ 500 ms
- โปรแกรมแบบง่ายแต่ครอบคลุม

✅ ผลการทดสอบ

- มอเตอร์ทั้ง 4 ตัวทำงานครบและตรงทิศ
- หุ่นยนต์เคลื่อนที่ได้ทุกทิศ ไม่มีอาการสะดุด
- ระบบไฟฟ้าทำงานต่อเนื่อง ไม่มี delay
- ใช้งานได้แม้ไม่มี encoder → คุมด้วย vision อย่างเดียว

Target Detection App

จับภาพหน้าจอ Kovaak → ตรวจจับเป้าแบบ real-time → ส่ง dx, dy ไปควบคุมหุ่นยนต์



Capture Screen → YOLOv5 → คำนวณ dx, dy → ส่ง Serial ไปยัง ESP32

🧠 โครงสร้างโปรแกรม:

- Python 4 ไฟล์: main.py, capture.py, detect.py, distance.py
- เลือกหน้าต่างโดยพิมพ์ชื่อแอปลงใน Terminal และดึงภาพด้วย pygetwindow → จับภาพด้วย mss
- ใช้ YOLOv5 ตรวจจับเป้าทุก frame (real-time)

🎯 ฟังก์ชันหลัก:

- ตรวจจับเป้า → เลือกเป้าที่ใกล้ศูนย์กลางที่สุด
- แสดงระยะ, มุม (เวอร์ชันเก่า) → **เปลี่ยนเป็น dx, dy**
- เปลี่ยนโหมดได้ด้วยปุ่ม W/E/R:
 - W: Idle
 - E: Aim+ Hold
 - R: Aim + Click

🔄 อัปเดตล่าสุด:

- เปลี่ยนจากส่ง "angle" → เป็น "dx, dy"
- ให้ PC คำนวณ Inverse Kinematics → ESP32 แคร่รับ wheel speed
- ลดความซับซ้อนและ latency ของระบบ

Intro → Design → Testing → Wrap-up

Clicking Control Logic

จำลองคลิกด้วย Python → ไม่ต้องใช้ ESP32 → ควบคุมง่ายกว่า

🔧 ปัญหาเดิม:

- แพลนแรกให้ **ESP32** ส่งคลิกผ่าน **Bluetooth** HID
- → ยุ่งยาก: ต้องจับคู่ Bluetooth, delay , ทดสอบลำบาก
- ต้องเปิดหน้าต่าง **"Aimbot View"** ไว้เสมอเพื่อให้โปรแกรมอ่านปุ่มได้
- → ถ้าสลับกลับเข้าเกมจะ **กดปุ่มควบคุมไม่ได้**

💡 แนวทางใหม่:

- ใช้ Python ไลบรารี pynput.mouse → **จำลองคลิกเองจากฝั่ง PC**
- เปลี่ยนระบบอ่านปุ่ม → ใช้ **keyboard** แบบ global hook
- → โปรแกรมจับปุ่มได้แม้ทำงานเบื้องหลัง

⚠️ ปัญหาใหม่:

- ปุ่ม W / E / R เดิม **ชนกับปุ่มเดิน-รีโหลดในเกมจริง**
- → กดเพื่อควบคุมระบบ → กลับกลายเป็นเดิน/ยิงในเกม

✅ ทางออก: เปลี่ยนมาใช้ปุ่ม **Z / X / C / V**

ปุ่ม	ความหมาย
Z	Quit (ออกจากโปรแกรม)
X	Mode 0: Idle (ไม่ทำอะไร)
C	Mode 1: Aim only (คลิกค้าง)
V	Mode 2: Aim + Pulse Click (ยิง)

🔄 ระบบ Pulse Click (Mode 2):

- ยิงเมื่อ "On Target" → click() → delay 26ms → release()
→ delay 27ms → ยิงใหม่
- **เลียนแบบการยิงของมนุษย์** → ไม่ดู robot จนเกินไป
- → ลดความเสี่ยงโดนเกมตรวจว่าเป็น Bot (Anti-cheat Friendly)

Serial Comm & Redesign Plan

ส่ง Distance/Angle ได้จริง แต่ไม่เวิร์คในเชิง Workflow → เปลี่ยนแผน

🔧 สิ่งที่ทำตอนแรก

ส่งข้อมูลจาก Python → ESP32 ด้วย PySerial

→ Distance, Angle, OnTarget, Mode

ESP32 รับผ่าน Serial.parseX() → ควบคุมมอเตอร์ตาม mode

e.g. mode 1 = เดินหน้า / mode 2 = ถอย

✅ Serial ใช้งานได้จริง (แสดงผลผ่าน Serial Monitor ได้ถูกต้อง)

⚠️ ปัญหาในการใช้งานจริง

- หากต้อง Upload โค้ดใหม่ ต้อง:
 - ปิดแอป → ปิด Serial
 - กด BOOT, → Upload ได้
 - เปิดแอปใหม่อีกครั้ง
- ทำซ้ำแบบนี้ทุกครั้งที่จูน PID → **เสียเวลามาก**

🔄 แนวทางใหม่: คำนวณทุกอย่างบนฝั่ง PC

- ให้ Python คำนวณความเร็วล้อ 4 ตัว (V_FL, V_FR, V_RL, V_RR)
- ส่งผ่าน PySerial → "23,-45,30,-40\n"
- ESP32 รับค่ามา → แค่สั่ง PWM ตามที่ได้
- **ไม่ต้องคิด logic เองอีกเลย**

✅ ข้อดีของแผนใหม่

- ไม่ต้อง Upload โค้ด ESP32 บ่อย
- จูน PID / Tuning / Optimization ได้จากแอปทันที
- พร้อมต่อยอดให้ฉลาดขึ้นในอนาคต เช่น logging, realtime plot

Minimum PWM & Mapping Logic

ทดสอบค่า PWM ที่ล้อเริ่มหมุน → พัฒนา logic map speed

🔍 ปัญหาเดิม

- ไม่มี feedback → วัดความเร็วจริงไม่ได้
- หากส่ง PWM < 140 → **ล้อไม่หมุนเลย** → หุ่น "นิ่ง" หรือ "กระตุก"

🧪 วิธีทดสอบ

- เขียน minspeedtest.py → กดคีย์บอร์ดเพิ่ม/ลด PWM แบบ real-time
- วัดจุดที่:
 - ☒ ล้อหมุนต่อเนื่องเองได้
 - ☒ ล้อหมุนถ้ามีแรงช่วยเบา ๆ
- ทดสอบครบ 4 ล้อ → สรุป Deadzone

✅ ผลลัพธ์

- เริ่มหมุนจริงที่ PWM $\approx$ **140**
- หากช่วยหมุน → ใช้ PWM $\approx$ 135
- เลือกใช้ค่ากลาง **140** เป็น baseline

🧠 นำผลไปพัฒนา Mapping Logic

- ฝั่ง PC ส่ง "Logic Speed" -100 → +100
- ESP32 map → PWM จริง (ผ่าน percentToPwm())

```
if v > 0:
    pwm = map(v, 1, 100, 140, 255)
if v < 0:
    pwm = -map(abs(v), 1, 100, 140, 255)
```



Integration: IK + Serial + Motion

เชื่อม YOLO → PID → IK → ส่ง Serial → หุ่นขยับจริงแบบ Real-time

🎯 Flow ของระบบ:

1. **YOLOv5 ตรวจจับเป้า** → ได้ค่าความคลาดเคลื่อน dx, dy
2. **PID Controller** คำนวณความเร็ว vx, vy จาก error (dx/dy)

3. Inverse Kinematics (IK):

- แปลง vx, vy → ความเร็วของล้อ 4 ตัว:
- $V_{FL} = V_y + V_x$
- $V_{FR} = V_y - V_x$
- $V_{BL} = V_y - V_x$
- $V_{BR} = V_y + V_x$
- $\omega = 0$ เพราะหุ่นยังไม่หมุนตัวเอง

4. Normalize Speed ให้ไม่เกิน $\pm 100\%$

5. ส่งผ่าน Serial:

- แปลงเป็นข้อความ "v0,v1,v2,v3\n" เช่น "20,-40,25,-30\n"
- ส่งผ่าน serial_sender.py



🧠 ESP32: รับข้อมูล → ควบคุม PWM

- ใช้ percentToPwm() แปลง logic speed → PWM จริง (deadzone > 140)
- ส่ง PWM ไปยัง DRV8833 ทุก 10 ms → ล้อเคลื่อนทันที

✅ ข้อดีของแนวทางนี้:

- ประมวลผลหนักฝั่ง PC → ESP32 มีหน้าที่เป็นแค่ตัวกลาง

YOLO → PID → IK → Serial → PWM

PID Tuning + Logging

ปรับ PID ด้วยสายตา → ยังไม่พอ → เขียน Logger วิเคราะห์พฤติกรรมจริง

🎯 เป้าหมายของ PID:

- ควบคุมหุ่นให้เคลื่อนเข้าหาเป้าบนหน้าจอ โดยใช้ dx, dy จาก YOLO เป็น error
- PID แต่ละแกนแยกอิสระ:
 - แกน X: Strafe (ซ้าย/ขวา)
 - แกน Y: เดินหน้า/ถอยหลัง

🔧 การเลือก Setting :

- ใช้ DPI 800 (ค่า fix ของเมาส์)
- ใช้ 50 cm/360° sensitivity:
 - ลด overshoot
 - หุ่นเคลื่อนที่ช้าแต่แม่นยำ
 - ใช้ความเร็วช้าลงเพื่อชดเชยแรงบิดต่ำของมอเตอร์
- Resolution เกม: 1920×1080 (มาตรฐานสำหรับ Competitive Gaming)

👁️🗨️ เริ่มต้นด้วยการจูนแบบ manual:

1. ตั้งค่าเริ่ม $K_p \neq 0$, $K_i = 0$, $K_d = 0$
2. ทดสอบกับเป้านิ่ง → overshoot
3. เพิ่ม K_d → ลดความแรงตจนถึงเป้า
4. เพิ่ม K_i หากเป้าไถลแล้วไม่เข้าเป้า
5. ทดสอบกับ Scenario ที่เร็วขึ้นเรื่อย ๆ

⚠️ ปัญหาที่พบบ่อย:

- พัดลื่น: หุ่นลื่น overshoot แม้สั่งหยุด
- มอเตอร์ราคาถูก (1:48 Gear): Torque ต่ำ → เคลื่อนที่ไม่ละเอียด
- Deadzone: PWM ต่ำกว่า 140 ล้อไม่ขยับ → ทำให้เสียความเร็วต่ำยาก
- ไม่มี Encoder: วัด error ได้แค่จากภาพ ไม่มี feedback จริง

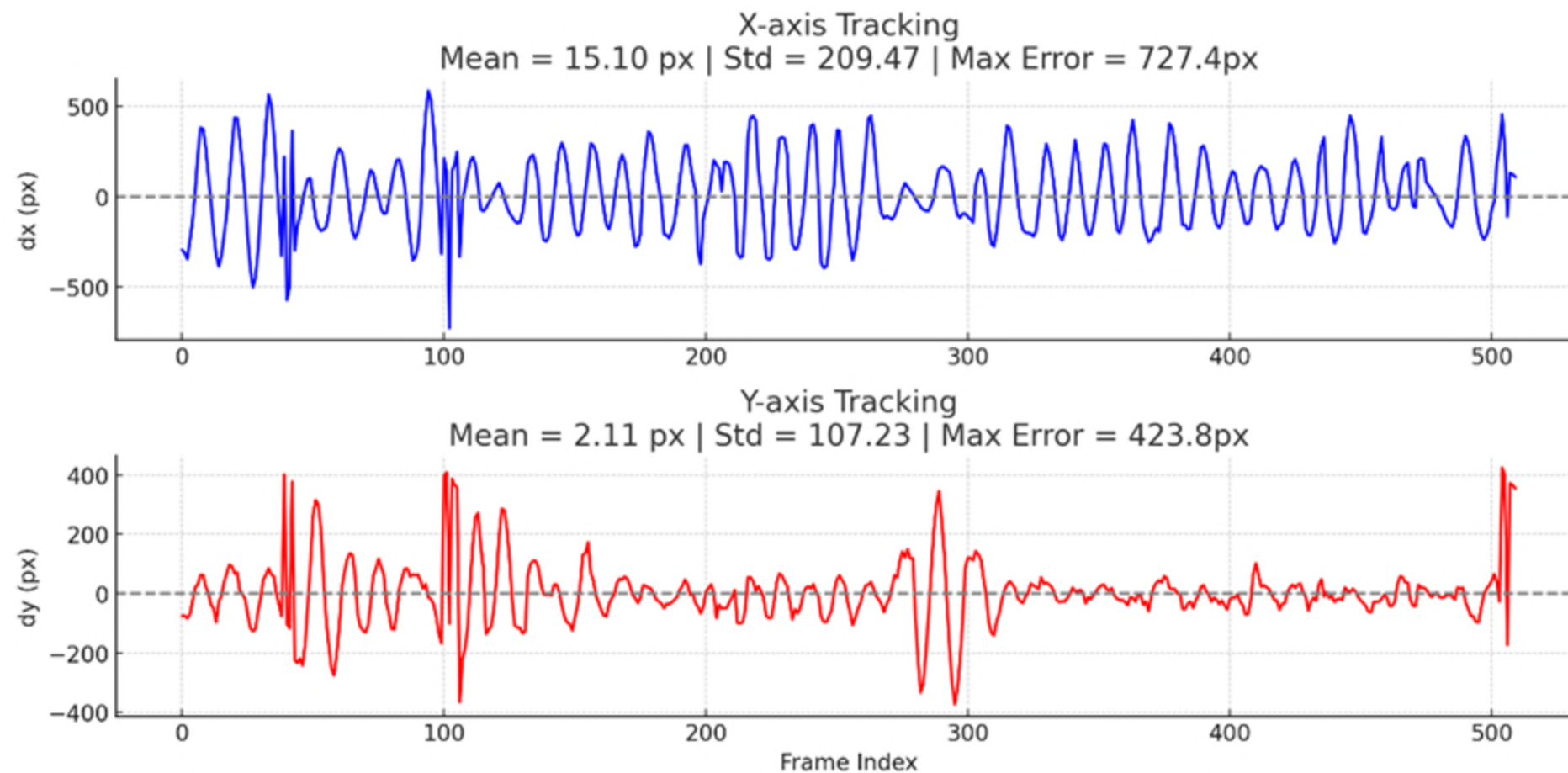


🔍 แนวทางแก้: เขียน Logger → Data-Driven Tuning

- ✨ ใช้ logger.py → เก็บ dx, dy, mode ทุกเฟรม
- 📊 ใช้ plotter.py → วิเคราะห์พฤติกรรมจากกราฟ

PID Tuning + Logging

ปรับ PID ด้วยสายตา → ยังไม่พอ → เขียน Logger วิเคราะห์พฤติกรรมจริง



🧠 ค่าที่ใช้หลังจูนเสร็จ (baseline):

$Kp_X = 0.18$ $Ki_X = 0.5$ $Kd_X = 15.0$

$Kp_Y = 0.13$ $Ki_Y = 0.3$ $Kd_Y = 7.0$

- ทดสอบใน Scenario ที่ความเร็ว ~70% ของเกมจริง
- ใช้งานได้ในระดับ “พอใช้” เหมาะเป็นจุดเริ่มต้น

📈 ผลการวิเคราะห์กราฟ (เฉพาะ mode = 1):

X-axis

- Mean Error: 15.10 px
- Std Dev: 209.47 px
- Max Error: 727.4 px
- → ⚠️ Overshoot นึก และไม่เสถียรในแนว Strafe

Y-axis

- Mean Error: 2.11 px
- Std Dev: 107.23 px
- Max Error: 423.8 px
- → ⚠️ เสถียรกว่าแกน X แต่ยังมี jitter

💬 ข้อเสนอแนะจากการวิเคราะห์ (Future Plan):

ปัญหา	แนวทางที่เสนอ
ลื่น, หยุดไม่ทัน	เปลี่ยนพื้น test ใหม่
มอเตอร์ Torque ต่ำ	ใช้มอเตอร์ gear สูงขึ้น (เช่น 1:100)
ไม่มี feedback	เพิ่ม Encoder → PID แบบ Closed-loop
PID ไม่มีความทนต่อความเบี่ยงเบน	เพิ่ม deadband, anti-windup logic

Testing Phase

Software + Hardware + Integration Test

Object Detection Evaluation

ประเมินความสามารถ YOLOv5 บนวิดีโอ Kovaak จริง

 วัตถุประสงค์ของการทดลอง
ประเมินว่า โมเดล YOLOv5 ขนาดต่าง ๆ สามารถตรวจจับ Robot
ในวิดีโอเกม Kovaak ได้แม่นยำแค่ไหน
เพื่อใช้เลือก ขนาดโมเดลที่เหมาะสมกับการใช้งานจริงแบบ Real-time

 ตัวแปรการทดลอง

ประเภทตัวแปร	รายละเอียด
ต้นทาง (IV)	โมเดล YOLOv5: s_old, s_new, m, l, x Input ขนาด 640×640
ตาม (DV)	อัตราการตรวจจับ (%), ค่า confidence เฉลี่ย, วัตถุที่ตรวจพบ
ควบคุม (CV)	Resolution 1920×1080, FPS 12, ความยาว 10 วิ, skip_frames = 1

 วิธีการทดลอง
Manual Counting (Baseline)
ใช้ Python แสดงเฟรมทีละภาพ → นับด้วยมือ
เพื่อเทียบกับการตรวจจับอัตโนมัติ
Automated Testing
ใช้ modeltest.py → โหลดวิดีโอ → รับ YOLO frame-by-frame
ตรวจว่าในแต่ละเฟรมตรวจจับได้หรือไม่ และแม่นยำแค่ไหน

 Scenario ทั้ง 5 ที่ใช้ทดสอบ
(ทุกวิดีโอ: 10 วินาที, 12 fps, รวม ~119–120 เฟรมต่อชุด)

Scenario	ความยาก	Notes
Colosseum No Shooting	กลาง	วิ่งเร็ว ไม่มี damage effect
CLS Click Robots	ง่าย	เป้าเด่น ชัด ไม่มี overlap
RoboTS180	ยาก	เป้าหลายตัว วิ่งชนกัน ช้อนกัน
Cata IC Fast Strafes	กลาง	มี visual effect / กล้องเปลี่ยน
OW Close Fast (Invincible)	ยาก	กล้องไถลมาก → หลุด bounding box

Object Detection Evaluation

ประเมินความสามารถ YOLOv5 บนวิดีโอ Kovaak จริง

 ผลการทดสอบแบบสรุป (Detection Accuracy %)

Scenario	s_old	s_new	m	l	x
CLS Click Robots	73.1	74	93.3	100	100
Cata IC Fast Strafes	79.8	26.9	31.1	58	38.7
Colosseum No Shoot	60	92.5	98.3	98.3	99.2
OW Robot (Close)	0.8	98.3	100	100	100
RoboTS180	84.9	37	19.3	90.8	37.8

● Confidence Score (เฉลี่ย):

s_old = 0.52

s_new = 0.43

m = 0.47

l = 0.57

x = 0.54

วิเคราะห์ผล

- YOLOv5l / x ทำงานได้ดีที่สุดโดยรวม โดยเฉพาะฉากที่มี object overlap หรือ background สุกวน
- s_new ดีกว่า s_old ในหลายจุด → แสดงว่า Dataset v2 ช่วยให้ generalize ดีขึ้น
- YOLOv5m มีอาการ overfit ชัดเจน ตรวจได้ดีในบางฉาก แต่พลาดมากในฉากที่ไม่คุ้น
- ฉาก RoboTS180 กับ Cata IC Fast Strafes → ทำท่าย model มากเพราะมีการ overlap, damage, มุมกล้องใหม่

ข้อสังเกตสำคัญ

- mAP จาก validation ไม่เพียงพอ → ต้องดู performance จากวิดีโอจริงร่วมด้วย
- การเลือกโมเดลที่ดีที่สุด ควรดูร่วมกับ latency และ FPS (ต่อจากหัวข้อนี้)

Latency & FPS Impact

ทดสอบ Latency + FPS → ประเมินความเหมาะสมโมเดลในงาน Real-time

🎯 วัตถุประสงค์ของการทดลอง

วัด **เวลาแฝง (Latency)** และ **ผลกระทบ FPS** ของโมเดล YOLOv5 ขนาดต่าง ๆ เพื่อใช้ตัดสินใจเลือกโมเดลที่เหมาะสมกับระบบ Aimbot แบบ เฟรมต่อเฟรม

🧪 ตัวแปรการทดลอง

ประเภท	รายละเอียด
ต้น	ขนาดโมเดล: s_old, s_new, m, l, x
ตาม	Mean / Min / Max Latency (ms), FPS ของเกม
ควบคุม	Resolution: 1920×1080, FPS: 12, V-sync: Off, Scenario เดิม, Python + PyTorch 2.1 (CUDA 12.1), RTX 3060 Laptop

🕒 ผลทดสอบ Inference Latency (ms/frame)

Model	Mean	Min	Max	FPS ~
s_old	15.5	11.6	185.7 ¹	~64 FPS
s_new	15.6	11.5	34	~64 FPS
medium	18.9	14	90.5	~53 FPS
large	23	17.5	92.5	~43 FPS
x	30.8	26	109.5	~32 FPS

¹ เกิดจาก spike ชั่วคราวจาก background process

📌 ผลทดสอบ In-Game FPS Impact (Kovaak Game)

Model	GFLOPs	Game FPS	Aimbot View (cv2)
s_old	15.8	274	กระตุก หลุดเป้าบ้าง
s_new	15.8	267	นิ่งกว่าเดิม ไม่ติดบางจังหวะ
medium	48.2	243	เสถียรขึ้น, ตอบสนองดี
large	108.2	252	แม่น นิ่ง, UI ช้าลงนิดหน่อย
x	204.6	247	แม่นสุด แต่ overlay เริ่ม lag

📝 FPS baseline: ~270–290 (ไม่เปิดโมเดล)

Latency & FPS Impact

ทดสอบ Latency + FPS → ประเมินความเหมาะสมโมเดลในงาน Real-time

🔍 วิเคราะห์ & สรุปเชิงเทคนิค

- ✅ s_old / s_new: ความเร็วสูง (~64 FPS), ใช้งาน real-time ได้
- ▶ s_new มี latency สม่่าเสมอกว่า → เหมาะกว่า s_old
- 🟡 medium: ตอบสนองเร็วพอ (~53 FPS) แต่มี spike ในเฟรมที่ซับซ้อน
- 🟠 large: แ่่นมาก (~43 FPS) แต่เริ่มกระทบความลื่น
- 🔴 x: แ่่นที่สุด แต่ latency สูงสุด (30.8 ms → 32 FPS) → ใช้ได้เฉพาะในงานที่ไม่เน้น fast-response

⚠️ ข้อสังเกตสำคัญ

- 🧠 Aimbot View ที่ใช้ cv2.imshow มีอาการ “แล็ค” ชัดขึ้นในโมเดลใหญ่
- 🎮 เกมยังลื่น (FPS >240) ทุกโมเดล → GPU เหลือพอ
- 👁 แต่ Overlay บนหน้าจอที่ใช้ควบคุมเล็ง → กระทบโดยตรงต่อ PID

💡 ค่าความมั่นใจ (Confidence Threshold) ที่เหมาะสม

→ 0.25

สมดุระหว่าง: Precision, Recall, GPU Load, และ False Positive

หน้า System Latency (ท้าย)

Full-System Test (Model + Movement)

เป็นการทดลองที่ทำลายคะแนนเฉลี่ยของผู้วิจัยมาก

ทดสอบ YOLOv5 ทั้ง 5 ขนาด → ควบคุมหุ่นจริงใน Kovaak

🎯 วัตถุประสงค์ของการทดลอง

- ทดสอบระบบแบบ End-to-End → จากการตรวจจับเป้าจริง → สิ่งเคลื่อนที่ → คลิกเป้าจริงใน Kovaak
- เปรียบเทียบผลของโมเดลแต่ละขนาด (Score / Accuracy)

📱 ตัวแปรการทดลอง

ตัวแปรต้น (IV)

-ขนาดของโมเดล YOLOv5 ที่ใช้ (s_old, s_new, m, l, x)

ตัวแปรตาม (DV)

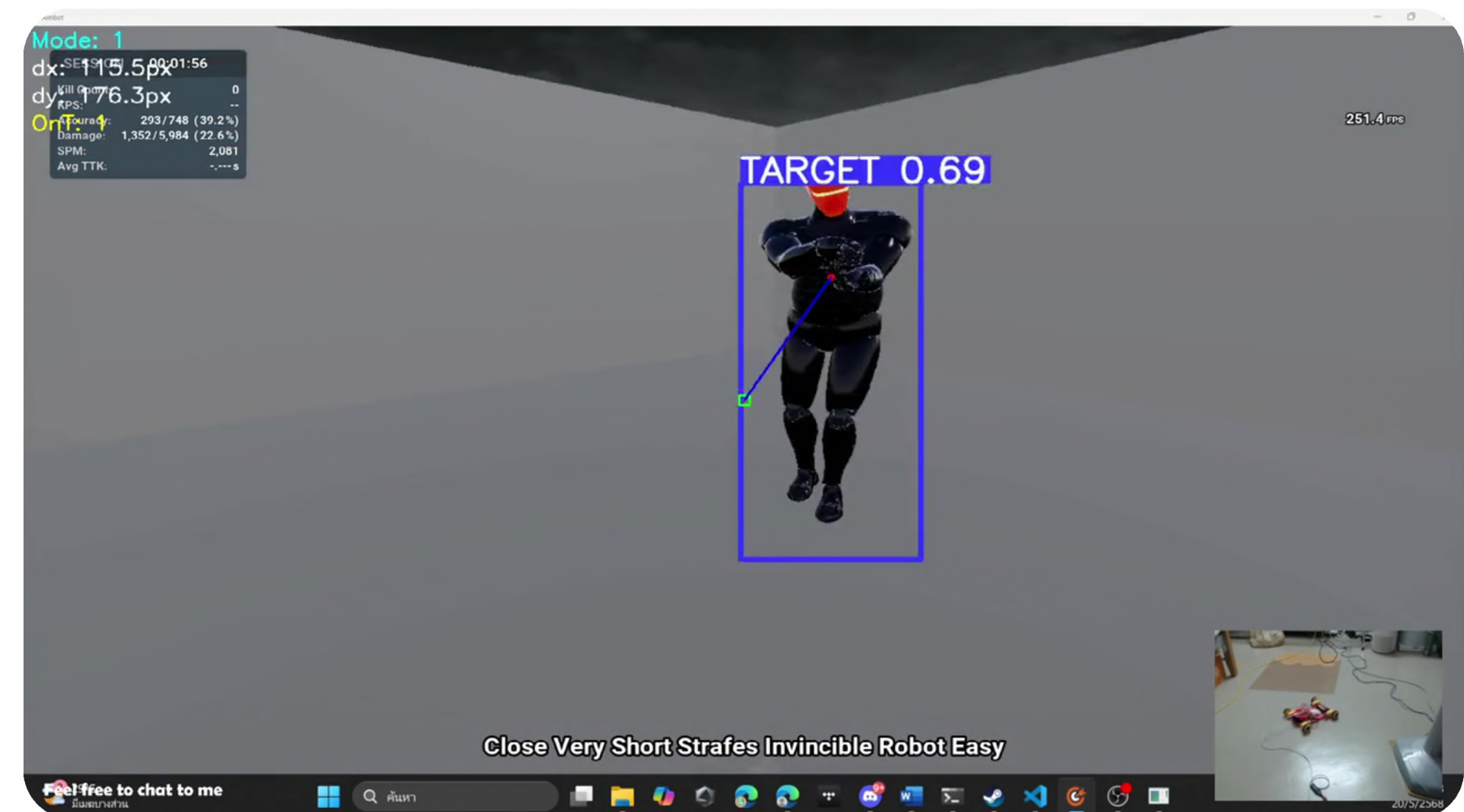
-คะแนน (Score) และ Accuracy (%) ที่ได้จากเกม Kovaak

ตัวแปรควบคุม (CV)

- สเปกเครื่องทดสอบ: ASUS TUF A15 (RTX 3060)
- ความละเอียด: 1920×1080 (V-sync OFF)
- Software/Framework: PyTorch 2.1, YOLOv5
- PID controller: ใช้ค่าที่จูนไว้ในหัวข้อ 3.6.5
- Scenario: ใช้ 3 แบบเดิมกับทุกโมเดล
- Mode คลิก: Pulse Click (Mode 2)
- จำนวนรอบ: 1 รอบ/โมเดล/Scenario (เพื่อควบคุมสภาพแวดล้อม)

📱 รายการ Scenario ที่ใช้:

Scenario	ลักษณะ	ใช้ทดสอบ...
Warzone	เป้า dynamic, ย่อหลบ	ความสามารถสูงสุดของระบบ
Kindaclose Long	ระยะไกล, ไม่ย่อ	ความแม่นยำ motion + ตรวจจับไกล
Easy 25%	ตัวใหญ่, ช้า	baseline ระบบง่าย



Full-System Test (Model + Movement)

ทดสอบ YOLOv5 ทั้ง 5 ขนาด → ควบคุมหุ่นจริงใน Kovaak

✓ ผลการทดสอบและวิเคราะห์ระบบรวม (Real-Scenario Model Test)

Model	Warzone (หนักมาก)	Long Strafes (กลาง)	Easy 25% (ง่าย)
Small Old	10 pts / 22.7% 🎯	36 pts / 100% ✓	108 pts / 50.0% ⚠️
Small New	314 pts / 23.6% ✓	540 pts / 29.7% ⚠️	828 pts / 36.8% ⚠️
Medium	198 pts / 14.9% ⚠️	306 pts / 11.3% ⚠️	324 pts / 9.5% ❗
Large	122 pts / 10.9% ❗	666 pts / 19.9% ⚠️	504 pts / 11.8% ❗
X-Large	30 pts / 6.4% ❗	270 pts / 14.2% ❗	198 pts / 7.3% ❗

- ค่าเฉลี่ยผู้เล่นในเกม
- Warzone 8,587 pts
 - Long Strafes 15,192 pts
 - Easy 25% 11,952 pts

📍 ด้าน Detection

ขนาดโมเดลมีผลต่อ “ความสม่ำเสมอ” ของการตรวจจับ: medium ขึ้นไปทำได้ดีตลอด แต่ dataset มีผลมากกับความแม่นยำเชิงมุม (เช่น เป้าหันข้าง, ย่อ) ยังไม่มี logic ติดตามเป้าหมายนอกเฟรม (หุ่นกลิ้งต้องใช้มือ)

📍 ด้าน Motion Control

ระบบ PID ตอบสนองไม่ทันใน scenario ที่เร็ว → เกิด overshoot small_new คุมได้แม่นยำที่สุดเพราะมี latency ต่ำ medium-x มี motion lag ชัดเจน → ยังไม่เข้าแม้ detect ได้

🧠 วิเคราะห์ผล: ตามโมเดล

♦ Small Old
ตรวจจับเป้าแบบ dynamic ไม่ได้เลย โดยเฉพาะเป้าที่ย่อหรือเคลื่อนไหวเยอะ
ทำคะแนนได้บ้างในฉากเป้าคงที่ (Long Strafes) แต่ไม่สอดคล้องกับการใช้งานจริง
แสดงว่า dataset เก่ามี bias และไม่ครอบคลุมเพียงพอ

♦ Small New
ตรวจจับเป้าได้ดีกว่า old โดยเฉพาะกรณีเป้าอยู่กลางจอ
แต่ยังหลุดบ่อยในฉากที่เป้าหันข้าง → dataset v2 ยังไม่หลากหลายเพียงพอ
ควบคุม motion ได้ดีขึ้นเมื่อเทียบกับ old

♦ Medium
ตรวจเจอเป้าเกือบทุก frame → แสดงว่า model เสถียรพอสำหรับ real-time
แต่ accuracy ต่ำมาก เพราะ “overshoot” บ่อย → PID ที่จูนไว้ยังไม่ตามทัน
การเคลื่อนไหวเร็ว

♦ Large
ตรวจเจอครบ frame เช่นกัน → แม่นยำกว่า medium เล็กน้อย
แต่ระบบเริ่ม lag และ PID ที่ไม่ละเอียดพอทำให้ยิงพลาดแม้ detect ได้
ชัดเจนว่าระบบเริ่มรับภาระ latency เพิ่มขึ้น

♦ X-Large
ตรวจเจอทุกเป้าแต่ “กระตุกหนักมาก” → Aimbot View เหลือ ~20 FPS
ทำให้ระบบ PID ทำงานไม่ทัน → แม้ detect ได้แต่ motion lag จนยิงพลาด
แสดงว่าระบบของเรายังไม่พร้อมสำหรับโมเดลขนาดใหญ่มาก

PID Behavior in Real Scenarios + System Summary

🎯 วัตถุประสงค์ของการทดลอง

เพื่อประเมินว่า PID ที่จูนไว้ใน Sandbox ยังทำงานได้ดีหรือไม่เมื่อเจอกับ Scenario จริงที่ dynamic สูง เช่น target teleport, move fast, หรือหลุด frame

🔧 ตัวแปรการทดลอง

ประเภทตัวแปร	รายละเอียด
ต้น	ลักษณะของ Scenario ในเกม Kovaak (close/fast, blink, etc.)
ตาม	ค่า error (dx/dy), การเข้าเป้า, overshoot
ควบคุม	Model = YOLOv5m Hardware = ASUS TUF A15, RTX3060 Resolution = 1920×1080 PID = ค่าจากหัวข้อ 3.6.5 Mouse sensitivity = 50 cm/360°

🔧 วิธีทดลอง (How We Measured It)

1. ใช้ main.py รันระบบ Object Detection → PID → ส่งไป ESP32
2. ใช้ logger.py บันทึก dx, dy, mode ทุกเฟรมที่ทำงานใน Scenario จริง (Mode 1)
3. เมื่อ Scenario จบ → export log เป็น .csv
4. ใช้ plotter.py วาดกราฟ dx(t), dy(t) เพื่อดูลักษณะพฤติกรรม PID
5. วิเคราะห์ error → ตรวจสอบ steady-state, overshoot, oscillation

📊 กรณที่ 1: Close Short Strafes (Fast but Predictable)

พฤติกรรมจากกราฟ:

✅ Error ลดลงเรื่อย ๆ → PID ตอบสนองทัน

⚖️ ไม่มี jitter หรือ overshoot รุนแรง

🔄 dx/dy เปลี่ยนช้า → เหมาะกับการประเมิน PID



🔍 วิเคราะห์:

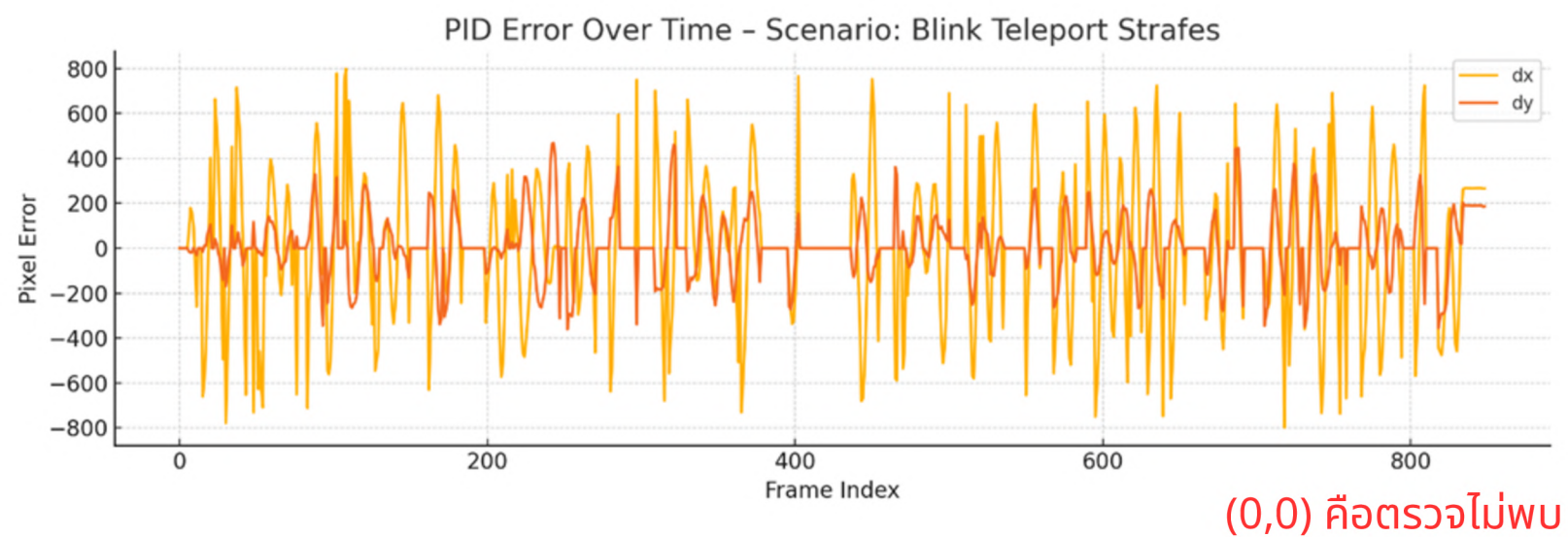
- ค่า mean ใกล้ 0 → หุ่นเข้าใกล้เป้าโดยรวม
- แต่ std สูงมาก → บางช่วง “สวิงแรง”
- สาเหตุ:
 - ⚙️ หุ่นมี inertia + deadzone (PWM < 140 ไม่หมุน)
 - เป้าเปลี่ยน speed → PID lag แล้วจึง overshoot
 - ไม่ได้ทำ anti-windup logic → ทำให้หยุดไม่ทัน

PID Behavior in Real Scenarios + System Summary

⚠️ กรณีที่ 2: Mid Strafes + Blink (Discontinuous Tracking)

พฤติกรรมจากกราฟ:

- ✗ มีช่วง $dx/dy = 0$ ค้าง → แปลว่า model ไม่เจอเป้าเลย
- ▲ ทุกครั้งที่เป้ากลับมา → error spike เกิดทันที
- 🔄 ระบบวิ่ง overshoot ทุกครั้ง → สั่น, กระชาก, หลุดเป้า



🔍 วิเคราะห์:

- แม้ mean ใกล้ 0 แต่ std และ max สูงมาก
- PID ทำงานได้ เฉพาะตอนมี input
- ทุกครั้งที่ blink → ไม่มี dx/dy ให้ใช้ → หยุดนิ่ง
- พอ model กลับมา → error พุ่งแรง → PID วิ่ง “พรวด” → overshoot กันที

📌 สรุปเปรียบเทียบ

ปัจจัย	Close Short Strafes	Blink Strafes
Continuity	✓ ตรวจเจอตลอด	✗ ขาด input หลายเฟรม
PID Stability	✓ เสถียร	✗ แกว่งรุนแรง
Overshoot	⚠️ มีแต่ควบคุมได้	▲ รุนแรงและถี่

🔧 แนวทางปรับปรุง (จากผลวิเคราะห์)

เพิ่ม deadzone, anti-windup ใน PID

เสริม predictive tracking: estimate ความเร็วเป้า (velocity)

→ ถ้า input หายยังคงเคลื่อนได้

ใส่ logic “รอ confirm เป้ากลับมา” → smooth error spike

Wrap - Up

Summarize Everything

สรุประบบโดยรวม (System Summary)

 **เป้าหมาย:** พัฒนา Aimbot ภายภาพที่ตรวจจับเป้า → คำนวณ → ควบคุมหุ่นยนต์จริงเพื่อเล็งและคลิกในเกม FPS แบบอัตโนมัติ

ระบบประกอบด้วย:

YOLOv5: ตรวจจับเป้าแบบเรียลไทม์จากหน้าจอเกม

PID Control: คำนวณความเร็วจาก error → ส่ง IK

ESP32 + Mecanum: ขับล้อจริง เลื่อนเมาส์จริง

Clicking: ใช้ pynput คลิกอัตโนมัติเมื่อเข้าเป้า

ขอบเขต:

-  ตรวจจับจริงจาก Kovaak
-  ทดลองหลายโมเดล YOLOv5
-  จูน PID ด้วย log → ใช้งานจริง

ข้อจำกัด:

-  ไม่ฝึก YOLO จากศูนย์
-  ไม่ใช้งานกับเกมเชิงพาณิชย์
-  ไม่พัฒนา system ป้องกัน cheat

บทเรียนจากการพัฒนา (System Learnings)

🔍 การตรวจจับ (YOLOv5):

- **v5x**: แม่นยำมาก → แต่ latency สูง ควบคุมตามไม่ทัน
- **v5s**: เร็ว ลื่น → แต่ detect ขาดช่วง → PID idle
-  **v5m**: สมดุลที่สุด → ตรวจจับพอ, latency พอควบคุม

🧠 PID Controller:

- ถ้า detect ต่อเนื่อง → PID เสถียร
- ถ้า input ขาด → PID idle → error spike → overshoot
- ปัจจุบันยังเป็น **Vision Based** → ไม่มี feedback จากล้อ → ควบคุมไม่แม่นยำ

⚠️ ข้อจำกัดสำคัญ:

- พั่นลื่น → หยุดไม่ทัน → overshoot
- **ไม่มีระบบหมุนกลอง** → เป้าหลุดเฟรม = ระบบหยุด
- **Latency** รวม > **30ms** → ควบคุมตามไม่ทันเกมจริง
- มอเตอร์ราคาถูก (**Yellow TT**) → แรงบิดต่ำ, ควบคุมละเอียดไม่ได้, เกิด jitter ง่าย

THANK YOU

โมเดลใหญ่ แม่ย่าสูง
สัญญาณขาด หยุตไปก็
ถ้าเต็มเฟด แทรกฟีดแบ็ก
หมูนกล่องได้ ไล่เป้าไว

แต่วิ่งจูง ไม่ทันพอ
ควบไม่พอ เกิดลิ้นไถล
มอเตอร์แม่่น สั่งล่อไป
หวังจะได้ ระบบจริง

Q & A